

LogDesk — Production-Ready Investor Demo Build Specification

You are a senior full-stack engineer building a complete, production-quality application called **LogDesk**.

The goal is to demonstrate real-time digitization of physical logbooks using intelligent document processing and live synchronization.

This is not a prototype, mockup, proof-of-concept, placeholder application, or starter template.

Build a fully functional application that can be deployed immediately to Vercel and used during a live investor demonstration.

Product Overview

LogDesk converts handwritten or printed logbook pages into structured digital records.

A user scans a logbook page from a mobile phone.

The image is uploaded to a Vercel serverless function.

A backend document-processing engine extracts and intelligently structures the data.

The extracted records are written into Firebase Realtime Database.

A viewer dashboard running on a laptop updates instantly as records arrive.

The audience sees physical records become digital in real time.

The user-facing experience should present LogDesk as a seamless document digitization platform. All intelligent extraction and processing technologies remain entirely behind the scenes and are never referenced in the interface, branding, status messages, page content, browser-visible configuration, client-side code, network responses, metadata, or user-facing documentation.

Only developers working on the backend infrastructure should be aware of the underlying processing architecture.

Core Principles

The application should feel:

- Fast
- Corporate
- Professional
- Reliable
- Modern
- Investor-ready
- Mobile-first for scanning
- Large-screen optimized for viewing

Design language:

- Blue corporate theme
 - Clean typography
 - Minimal visual noise
 - Smooth transitions
 - Subtle shadows
 - Professional spacing
 - High perceived performance
-

Architecture

Frontend

Vanilla:

- HTML
- CSS
- JavaScript

No frontend framework.

No build step required.

Everything should work immediately after deployment.

Backend

Vercel Serverless Functions

Used for:

- PIN verification
 - Session issuance
 - Document processing
 - Cloudinary upload
 - Firebase writes
-

Database

Firebase Realtime Database

Purpose:

- Real-time synchronization
 - Live dashboard updates
-

Processing Engine

Gemini 2.5 Flash

Used internally on the backend for:

- OCR
- Table understanding
- Handwriting interpretation
- Intelligent column inference
- Structured data generation

Implementation details remain server-side only.

No frontend component, browser inspection, network payload, UI text, or public-facing documentation should reveal the use of Gemini, AI models, machine learning, generative AI, LLMs, or any related terminology.

Image Archiving

Cloudinary

Every uploaded image must also be saved to Cloudinary.

Images are retained permanently.

Image URLs are stored alongside extracted records.

Auto-generated filenames.

Organized automatically by date folders:

YYYY/MM/DD

Example:

2026/06/15/image_XXXXX

Application Routes

Route 1

/upload

Purpose:

Mobile scanning interface.

Primary user:

Operator.

Route 2

/viewer

Purpose:

Live dashboard.

Primary user:

Audience.

Security Model

Two separate PIN systems.

Upload PIN

Environment Variable:

UPLOAD_PIN

Length:

4 digits

Purpose:

Unlocks upload interface.

Only operator should know this PIN.

Viewer PIN

Environment Variable:

VIEWER_PIN

Length:

4 digits

Purpose:

Unlocks viewer dashboard.

Can be shared with audience.

Authentication Flow

When a PIN is submitted:

1. Browser sends PIN to server.
2. Server validates against environment variable.
3. Server issues signed session token.
4. Token stored in sessionStorage.
5. Access granted.

Session remains active for 10 minutes after successful authentication, after which re-authentication is required. Closing the tab also ends the session immediately.

Environment Variables

UPLOAD_PIN=

VIEWER_PIN=

SESSION_SECRET=

GEMINI_API_KEY=

FIREBASE_DB_URL=

FIREBASE_SECRET=

CLOUDINARY_CLOUD_NAME=

CLOUDINARY_API_KEY=

CLOUDINARY_API_SECRET=

Firebase Structure

logs/
log_id/
imageUrl
headers
rows
createdAt

Upload Screen Requirements

Initial State

User sees:

LogDesk logo

Title:

"LogDesk"

Subtitle:

"Digital Logbook Processing"

PIN entry card.

4-digit input.

Unlock button.

After Unlock

Display:

Camera upload interface.

Mobile optimized.

Large upload area.

Accept:

- Camera capture
- Image files

Supported:

- JPG
- JPEG
- PNG

- WEBP
-

Upload Flow

User selects image.

Status updates:

Uploading...

Processing...

Saving...

Completed ✓

Status line remains simple.

Professional.

Business-oriented.

No technical terminology.

No references to AI, OCR, machine learning, Gemini, automation engines, or extraction models.

Visual Design

Clean.

Minimal.

Blue corporate styling.

Centered layout.

Excellent spacing.

Responsive.

Server Processing Pipeline

When image arrives:

Step 1

Validate upload session.

Step 2

Upload image to Cloudinary.

Receive permanent URL.

Step 3

Process document and extract structured records.

Step 4

Store results in Firebase.

All processing logic remains server-side.

Document Extraction Instructions

The backend processing engine should:

1. Analyze the entire page.
2. Detect all table columns.
3. Detect all rows.
4. Infer unclear headers from row contents.
5. Create intelligent header names when necessary.
6. Normalize inconsistent spacing.
7. Preserve original meaning.
8. Return structured records.
9. Return valid JSON only.

Expected response format:

```
{  
  "headers": [  
    "Name",
```

```
    "Reference Number",
    "Time In",
    "Time Out",
    "Host",
    "Remarks"
  ],
  "rows": [
    {
      "Name": "...",
      "Reference Number": "...",
      "Time In": "...",
      "Time Out": "...",
      "Host": "...",
      "Remarks": "..."
    }
  ]
}
```

Firestore Write Logic

After structured data is returned:

Write:

- headers
- rows
- imageUrl
- createdAt

to Firestore Realtime Database.

Generate unique IDs automatically.

Viewer Screen

Purpose:

Live audience display.

Optimized for laptop presentation.

Initial State

Show:

LogDesk logo

Title:

"Live Records"

Viewer PIN card

4-digit PIN input

Unlock button

Dashboard State

After unlock:

Display:

Spreadsheet-style live data workspace inspired by the familiarity of Google Sheets, featuring sortable columns, column resizing, row selection, inline cell editing, copy and paste support, keyboard navigation, basic filtering, search, frozen header row, automatic data updates from Firebase, real-time synchronization, and a clearly visible refresh button that reloads and refreshes the website on demand while maintaining a clean and professional enterprise dashboard experience.

Table Behavior

Columns generated dynamically.

Headers derived directly from incoming processed records.

No hardcoded schema.

Different logbook layouts must work automatically.

Live Updates

Use Firebase real-time listeners.

New data appears immediately.

No polling.

No refresh required.

Animation

When new rows arrive:

Slide upward smoothly into view.

Subtle fade-in.

Professional motion.

Consistent timing.

No flashy effects.

Dashboard Layout

Top Bar:

LogDesk logo

Live indicator

Export button

Refresh button

Live Indicator

Green pulse.

Text:

"Connected"

Updates automatically if Firebase disconnects.

Export Feature

Client-side Excel export.

Use SheetJS.

Single click.

Exports currently visible records.

Filename format:

logdesk-export-YYYY-MM-DD.xlsx

Table Features

Sticky header.

Horizontal scrolling on small screens.

Alternating row backgrounds.

Hover state.

Professional spacing.

Responsive typography.

Clouinary Support

Store image URL with every record batch.

Viewer should internally retain access to image references for future admin-panel integration.

Structure code so future admin application can easily display:

- Original scan
 - Extracted records
 - Batch history
-

Error Handling

Professional user-facing messages.

Examples:

"Invalid PIN"

"Upload failed. Please try again."

"Processing could not be completed."

"Connection restored."

"Connection lost. Reconnecting..."

Accessibility

Keyboard friendly.

Visible focus states.

Mobile friendly.

Screen-reader friendly labels.

Proper semantic HTML.

Performance

Fast initial load.

Lazy-load libraries where appropriate.

Optimize image uploads before transmission.

Prevent unnecessary Firebase reads.

Efficient DOM updates.

Smooth animations even with large tables.

Keep all processing logic server-side.

Avoid exposing implementation details through browser developer tools, frontend source code, public APIs, or client-side configuration.

File Structure

```
/
├── api/
│   ├── verify-pin.js
│   ├── process-image.js
│   └── utils/
│       ├── auth.js
│       ├── firebase.js
│       ├── gemini.js
│       └── cloudinary.js
├── public/
│   ├── upload.html
│   ├── viewer.html
│   ├── css/
│   │   └── styles.css
│   ├── js/
│   │   ├── upload.js
│   │   ├── viewer.js
│   │   ├── auth.js
│   │   └── excel.js
│   └── assets/
│       └── logo.svg
├── vercel.json
├── package.json
├── README.md
└── .env.example
```

Deployment

Application must deploy directly to:

Vercel

with no code modifications required beyond environment variables.

Include:

- Setup instructions
- Firebase configuration instructions
- Cloudinary configuration instructions
- Backend processing configuration instructions
- Local development instructions
- Production deployment instructions

Developer documentation may reference Gemini where necessary for setup and maintenance.

User-facing interfaces and experiences should never reference the underlying processing technology.

Expected Outcome

A user scans a physical logbook page from their phone.

Within seconds:

1. Image is archived in Cloudinary.
2. Structured records are extracted.
3. Firebase receives the records.
4. Viewer dashboard updates live.
5. New rows slide smoothly into the table.
6. Audience watches paper records become digital in real time.

The entire experience should feel polished, trustworthy, professional, and suitable for investor demonstrations, enterprise pilots, and future commercialization.

From the perspective of operators and viewers, LogDesk should appear as a seamless enterprise document digitization platform with intelligent processing occurring transparently in the background.